

- "i" points to valid elements of "nums1"
- "j" points to elements of "nums2"
- "k" points to the position where the next largest elements goes.
- Remaining elements of "nums1" do not need to be copied.
- Only remaining elements of "nums2" need to be copied.

▲ Complexity → Brute Force →  $O((m+n) \log(m+n))$  &  $O(1)$   
 Three Pointers →  $O(m+n)$  &  $O(1)$

### • Easy - #108 - Convert Sorted Array To Binary Search Tree

▲ Problem Statement → Given a sorted array of distinct integers, convert it into a height-balanced Binary Search Tree.

A height-balanced tree means the left & right subtrees have approximately equal heights.

▲ Main Solution → Use recursion & always choose the middle element of the current array range as the root.

Because the array is sorted:-

- Elements on the left of "mid" become the left subtree.

- Elements on the right of "mid" become the right subtree.

Choosing the middle keeps the tree balanced.

▲ Brute Force - Try different elements as roots & construct possible BSTs, then check which one is balanced.

This is inefficient & unnecessary because choosing the middle directly gives a balanced tree.

▲ Algorithm - 1 | Create a recursive helper function with "left" & "right" indices.

2 | If "left > right", return "nullptr".

3 | Find the middle index :-  $\text{mid} = \text{left} + (\text{right} - \text{left}) / 2;$

4 | Create a new node using `nums[mid]`

5 | Recursively Build :-

- Left subtree using "left" to "mid - 1".

- Right subtree using "mid + 1" to "right".

6 | Return the root node.

▲ Important Points -

- The array is already sorted

- Choosing the middle element as the root keeps the tree height - balanced.

- Left half becomes the left subtree.

- Right half becomes the right subtree.

- Recursion stops when "left > right".

|           |   |   |   |   |   |       |
|-----------|---|---|---|---|---|-------|
| M         | T | W | T | F | S | S     |
| Page No.: |   |   |   |   |   | YOUVA |
| Date:     |   |   |   |   |   |       |

Complexity → Main Solution →  $O(n)$  &  $O(\log n)$